

Hushh Protocol

Send silently.

A zero-knowledge payment rail for USDC on Arc. Hushh severs the on-chain link between who paid and who was paid, using a shielded note pool of Poseidon commitments and Groth16 proofs of Merkle membership, while presenting the user with nothing more complicated than sending a message to an `@username`.

NETWORK	CHAIN ID
Arc Testnet	5042002
ASSET	PROOF SYSTEM
USDC	Groth16 / BN254
STATUS	
Testnet, unaudited	

DOCUMENT	Hushh Protocol Whitepaper, v1.0
SUPERSEDES	Hushh Litepaper v1.0
SCOPE	Protocol specification, cryptographic construction, security analysis, and stated limits for the v1 shielded note pool
AUDIENCE	Protocol engineers, cryptography reviewers, integrators, and technically-inclined users evaluating real-world privacy risk
READING PATH	Sections 1–3 are conceptual. Sections 4–7 are the specification. Sections 8–10 are the security argument and its limits. Appendices carry parameters and notation.

ON THE HONESTY OF THIS DOCUMENT

A privacy protocol that overstates its guarantees is more dangerous than one that does not exist, because users calibrate real-world behaviour against its claims. Every property asserted here is stated with the assumption it rests on. Section 10 enumerates, without hedging, what Hushh does not protect. Where a number is an estimate rather than a measurement, it is labelled as such.

CONTENTS

01 Introduction

The transparency problem · why mitigations fail · contributions

02 Preliminaries

Notation · BN254 · Poseidon · Merkle accumulators · Groth16

03 Design goals and non-goals

Requirements · explicit exclusions · why Arc

04 Protocol specification

Parties · keys · notes · the four algorithms

05 The withdraw circuit

Relation · constraints · public inputs · soundness checks

06 On-chain design

BulletPool · root window · verifier · validation order · gas

07 Off-chain infrastructure

Indexer · backend · SDK · note encryption

08 Security analysis

Definitions · reductions · attack catalogue · trust matrix

09 Anonymity as a quantity

Effective anonymity set · timing decay · user guidance

10 Honest limits

What Hushh does not protect

11 Deployment

Parameters · addresses · services

12 Roadmap

Decentralising liveness before widening surface

13 Compliance posture

Viewing keys · selective disclosure · non-custody

14 Design principles and open problems

A Notation

Appendix

B Contract surface

Appendix

C Public input encoding

Appendix

D Parameter table

Appendix

R References

SECTION 01

Introduction

Every transfer on a public blockchain permanently publishes an edge: `sender → recipient`. For payroll, donations, remittances and vendor payments, that edge is not incidental metadata. It is the sensitive information. Hushh removes it.

1.1 The transparency problem

Public chains are transparent by construction. The property that makes settlement independently verifiable also makes every economic relationship globally readable and trivially clusterable. Transparency is not a bug to be patched; it is the mechanism by which the ledger is trusted at all. The question is therefore not how to make the chain opaque, but how to publish enough for verification and nothing more.

The cost of publishing more than that is concrete and falls differently across use cases:

USE CASE	WHAT THE CHAIN DISCLOSES, PERMANENTLY AND TO EVERYONE
Payroll	Each contributor's exact compensation, payment cadence, and affiliation with the paying entity, visible to competitors, counterparties, and the contributor's own peers. Salary confidentiality, a near-universal norm off-chain, is unachievable on-chain.
Donations	A permanent, enumerable list of supporters of a political, legal, or humanitarian cause, in jurisdictions where such a list may later be used against them. Unlike a leaked donor list, this one cannot be retracted.
Remittances	Recipients in a single corridor become a targetable cohort with known balances and predictable arrival times. The threat here is not analytical but physical.
Commerce	Supplier identities, order volumes, margins, and payment terms, disclosed to any counterparty before a negotiation begins.
Personal	Rent, medical payments, legal retainers, and support to family members, all attached to a long-lived address that is frequently linked to a real identity through a single exchange deposit.

The common structure is that the *relationship*, not the amount, is what leaks first and hurts most. An observer who learns that address A pays address B every fortnight has learned an employment relationship regardless of whether the amount is legible.

1.2 Why the usual mitigations fail

- **Fresh addresses per payment.** Defeated by funding-trail analysis. A new address must be funded before it can transact, and the funding transaction is precisely the edge the fresh address was meant to avoid. The technique displaces the link by one hop; it does not remove it.
- **Custodial intermediaries.** Reintroduce exactly the counterparty trust that on-chain settlement was adopted to remove, and concentrate a single deanonymisation target holding the complete mapping. The privacy is real but is held by someone else and is revocable by subpoena, breach, or policy change.
- **Mixers with variable amounts.** Amount correlation is the strongest passive heuristic available to an analyst. A withdrawal of 4,317.62 units shortly after a deposit of the same size is a link, whatever the cryptography in between.
- **Existing shielded pools.** Cryptographically adequate and operationally hostile: raw note strings the user must not lose, manual backups, no identity layer, no delivery mechanism, no recovery path. Their security model assumes an operator-grade user. Most payments are not made by operator-grade users, and a privacy tool used incorrectly is usually worse than none, because it produces confident false assurance.

THE GAP HUSHH TARGETS

Unlinkability that a non-cryptographer can operate correctly on the *first* attempt, denominated in the stablecoin people actually hold, on a chain whose dominant activity is stablecoin payments. The cryptographic lineage here is well understood and deliberately unoriginal. The contribution is the layer above it.

1.3 Contributions

Hushh is a shielded note pool with an identity and delivery layer built on top. Concretely, this document specifies:

1. A **fixed-denomination note pool** over ERC-20 USDC, in which deposit and withdraw share no on-chain field other than a Merkle root shared

with every other note in the tree (§4).

2. A **single withdraw circuit** that simultaneously enforces commitment derivation, Merkle membership, nullifier binding, and payout-address binding, with the recipient address fixed at proving time so that a proof observed in the mempool cannot be redirected (§5).
3. A **rolling 64-root acceptance window** that decouples proof generation from block timing, removing a race that otherwise penalises users precisely when the pool is busiest, without permitting unbounded root replay (§6.2).
4. A **strict safety/liveness split**: every off-chain component may stall the system and none may break it. The indexer and backend are specified as untrusted for both custody and unlinkability, and the specification states what each learns (§7, §8.5).
5. A **usability layer** of username resolution, an end-to-end encrypted note inbox, and browser-native proving, none of which is trusted with funds or secrets (§7.2).
6. A **quantitative treatment of the anonymity set**, including denomination partitioning and timing decay, so that users can reason about their actual exposure rather than a binary claim of privacy (§9).

What is deliberately *not* claimed: encrypted balances, mainnet readiness, a multi-party trusted setup, or a decentralised root poster. Each is roadmapped (§12) and each is listed as a present limitation (§10).

SECTION 02

Preliminaries

Nothing in this section is novel. It is included so that the specification in §4–§5 can be read without external references, and so that the security argument in §8 has named assumptions to rest on.

2.1 Notation

Let $\mathbb{F}$ denote the scalar field of BN254, of prime order

$$r = 21888242871839275222246405745257275088548364400416034343698204186575808495617$$

All circuit values are elements of $\mathbb{F}$. Byte strings that enter the circuit as field elements must be reduced mod r , and the specification is explicit about where that reduction happens (§5.3), because an implicit reduction is a classic source of soundness bugs. We write $x \leftarrow \$S$ for uniform sampling from a set, $H(\cdot)$ for the Poseidon permutation-based hash instantiated over $\mathbb{F}$, and $\parallel$ for concatenation. A negligible function in the security parameter λ is written $\text{negl}(\lambda)$.

2.2 BN254 and the pairing

BN254 (also bn128, alt_bn128) is a pairing-friendly Barreto–Naehrig curve with native EVM precompiles at addresses `0x06`, `0x07`, and `0x08` for addition, scalar multiplication, and the pairing check respectively. Its practical significance for Hushh is economic rather than cryptographic: an on-chain Groth16 verification over BN254 costs a fixed and modest amount of gas, whereas the same verification over BLS12-381 requires in-contract field emulation on chains without the corresponding precompiles, at a cost that makes per-payment shielding uneconomic.

The cost accepted in exchange is a security level. Following the Kim–Barbulescu improvements to the tower number field sieve, BN254's effective security against discrete logarithm attacks in the target group is estimated at roughly 100 bits rather than the 128 bits originally claimed. For a testnet payment rail with fixed denominations this is an acceptable margin; it is recorded here as a stated assumption rather than left implicit, and migration to a higher-security curve is a mainnet consideration (§12).

2.3 Poseidon

Poseidon is an algebraic hash function designed for efficient arithmetisation: its round function is a low-degree polynomial map over $\mathbb{F}$, so expressing it as R1CS constraints costs tens of constraints per permutation rather than the tens of thousands required by a bit-oriented hash such as Keccak-256.

The choice is load-bearing for the product, not merely an optimisation. A depth-20 Merkle path requires 20 hash evaluations inside the circuit. With Keccak, the resulting constraint count pushes browser proving time from seconds into minutes and, on mobile hardware, into failure. Poseidon is what makes client-side proving viable, and client-side proving is what makes the "server never holds a secret" invariant (§7) achievable rather than aspirational.

The cost accepted is maturity. Poseidon has substantially less cryptanalytic history than Keccak, and its security rests on the difficulty of algebraic attacks (Gröbner basis, interpolation) against a low-degree round function. Hushh uses the circomlib instantiation with the standard parameter sets for arity 1, 2, and 4.

2.4 Merkle accumulators

An incremental Merkle tree of depth d over Poseidon_2 accumulates up to 2^d leaves into a single root. Membership of a leaf is proven by supplying d sibling hashes and d direction bits; the verifier recomputes the root and compares. Hushh uses $d = 20$, giving a capacity of 1,048,576 notes, which is a deliberate over-provision: the tree is cheap to deepen at design time and expensive to migrate once notes exist in it.

Crucially, the direction bits must be constrained to be boolean inside the circuit. An unconstrained index bit permits a prover to interpolate between the two hash orderings and forge membership for a leaf that is not in the tree. This is stated again in §5.4 as a soundness-critical check because it is the single most commonly reported bug class in production Merkle circuits.

2.5 Groth16

Groth16 is a preprocessing zk-SNARK for R1CS with three properties Hushh depends on:

- **Constant proof size** (three group elements, 8 field elements when encoded for the EVM), independent of circuit size.
- **Constant verification cost**, dominated by a single pairing product check, again independent of circuit size. This is what makes withdraw gas predictable.
- **Knowledge soundness** in the algebraic group model: a prover producing an accepting proof for a statement can be extracted to a witness satisfying the relation.

The cost accepted is a *circuit-specific* trusted setup. The proving and verifying keys are derived from structured reference string material whose generation involves secret randomness ("toxic waste"). Retention of that randomness by any single party permits forging accepting proofs for false statements, which in Hushh means minting withdrawals against notes that were never deposited. It does *not* permit deanonymising users. This asymmetry matters and is developed in §8.4.

ASSUMPTION LEDGER

The security claims of §8 hold under: (A1) knowledge soundness of Groth16 over BN254; (A2) collision resistance of Poseidon over $\mathbb{F}$ at the arities used; (A3) honest generation and destruction of trusted setup randomness by at least one participant; (A4) discrete logarithm hardness in BN254 at the ~ 100 -bit level; (A5) IND-CCA security of the note encryption scheme (§7.4). Failures of A1–A3 are theft-class; failures of A5 are metadata-class. None of A1–A5 failing alone breaks on-chain unlinkability.

2.6 Related constructions

SYSTEM	RELATION TO HUSHH
Zerocash / Zcash	Origin of the note-commitment-plus-nullifier construction with encrypted amounts. Hushh adopts the structure and deliberately omits encrypted balances in v1, trading confidentiality for a smaller, auditable circuit.
Tornado Cash	The direct architectural ancestor: fixed denominations, a Merkle set of commitments, a nullifier revealed at withdraw. Hushh's departures are the binding of <code>recipientDigest</code> , <code>amount</code> , and <code>tokenHash</code> into the commitment (§5.5), an off-chain tree with a posted-root window (§6.2), and the identity and delivery layer (§7.2).
Railgun / Aztec	Full shielded pools with arbitrary encrypted amounts and in-pool transfers. Strictly stronger privacy, substantially larger circuits and trust surface. This is Hushh's v3 direction (§12), not its v1 claim.
Stealth addresses (ERC-5564)	Solve recipient-address unlinkability without a pool, but leave the funding edge and the amount fully public. Complementary rather than competing: Hushh's <code>recipientDigest</code> is designed to accept a stealth-derived digest, which is the P1 path to repeat-payment unlinkability.

SECTION 03

Design goals and non-goals

3.1 Goals

GOAL	STATEMENT
G1 · Unlinkability	No on-chain data connects a deposit transaction to the withdraw transaction that spends it, beyond membership in a root shared with every other note in the tree.
G2 · Non-custody	No party other than the holder of a note's secret can cause that note to be spent. This includes the contract owner, the indexer, the backend operator, and the protocol authors.
G3 · Liveness isolation	Every off-chain component is a liveness dependency and never a safety one. A total compromise of all off-chain infrastructure must not enable theft or deanonymisation.
G4 · First-attempt usability	A user with no cryptographic background completes a private payment correctly on the first try, addressing a human rather than a hex string, with no manual note handling.
G5 · Client-side secrets	Secrets, witnesses, and proofs are generated in the recipient's browser. No secret is transmitted, logged, or recoverable from anything the servers hold.
G6 · Calibratable risk	The system's limits are published in the same document as its claims, quantitatively where possible, so users can reason about actual exposure.

3.2 Non-goals for v1

These are excluded on purpose. Each is a real property that Hushh v1 does not have.

- **Amount confidentiality.** Amounts are quantised into fixed denominations, not encrypted. A 50 USDC note is a visible 50 USDC ERC-20 transfer. Privacy of the amount is statistical (everyone uses the same sizes), not cryptographic.
- **In-pool transfers.** Notes cannot be split, merged, or transferred between users without exiting the pool. Larger payments are composed from several notes.
- **Permissionless root posting.** A single relayer posts roots in v1. It cannot steal or deanonymise, but it can stall withdrawals (§8.5).
- **A multi-party trusted setup.** The demo ceremony is single-contributor. This is a prerequisite for mainnet, not for testnet (§10).
- **Automated social delivery.** Delivery over X or similar is copy-paste by design; automated DM delivery is a spam vector and is not built.
- **Programmatic payers and schedulers.** `deposit` remains callable by contracts, but no scheduling infrastructure is provided.

3.3 Why Arc

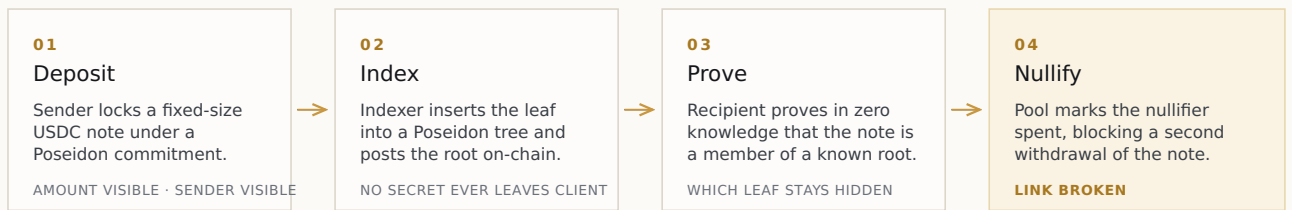
Deployment target is not incidental to a privacy design; it determines the size of the anonymity set, which is the quantity the entire guarantee scales with.

PROPERTY OF ARC	CONSEQUENCE FOR HUSHH
USDC as native gas and settlement asset	Users hold and reason in a single unit. There is no secondary gas token to acquire before making a private payment, and no volatile denominator complicating fixed-note sizing. The onboarding step most likely to leak metadata (buying gas) disappears.
Full EVM equivalence	Solidity contracts, Hardhat tooling, and the snarkjs-exported verifier pattern port directly. BN254 precompiles are available at predictable cost.
Predictable, low fees	Groth16 verification is a fixed, non-trivial gas cost. Fee predictability is what makes per-payment shielding economically ordinary rather than a luxury reserved for large transfers.
Fast, deterministic finality	The indexer posts roots promptly without deep reorg handling, keeping deposit-to-withdrawable latency short and simplifying the crash-recovery path (§7.1).
Payments-oriented ecosystem	The load-bearing point. A shielded pool's privacy scales with the number of unspent notes it holds. Deploying on a general-purpose chain where payments are a minority of activity produces a thin set; a chain concentrated on stablecoin settlement is where a USDC-denominated anonymity set can plausibly grow.

SECTION 04

Protocol specification

The protocol proper is four steps. Everything else in this document exists to make those four steps usable, or to state precisely what they do and do not guarantee.



THE ANONYMITY SET IS EVERY UNSPENT NOTE OF THE SAME DENOMINATION IN THE POOL
From the chain's perspective a withdrawal could have originated from any of them. The larger the pool, the weaker any correlation.

Figure 1. The four-step protocol lifecycle. Steps 1-4 are sufficient for privacy and insufficient for adoption; §7.2 describes the three components that carry the user experience, none of which is trusted with funds or secrets.

4.1 Parties

PARTY	ROLE	TRUSTED FOR
Sender	Generates the note secret, computes the commitment, deposits, and seals the note payload to the recipient's encryption key.	Correct generation of their own note. Nothing else.
Recipient	Decrypts the note, fetches a witness, proves membership in-browser, and submits the withdraw.	Custody of their own secret.
Pool contract	Custody, proof verification, nullifier set, token registry.	Safety. This is the only safety-critical on-chain component.
Indexer	Watches <code>Deposit</code> events, maintains the tree, posts roots, serves witnesses.	Liveness only. Cannot forge a leaf or learn a secret.
Backend	Wallet auth, <code>@username</code> resolution, encrypted inbox storage and delivery.	Liveness and metadata only. Holds ciphertext, never plaintext.
Contract owner	Token registry administration, verifier and root-manager replacement, pause.	Availability and correct upgrade. Has no withdrawal authority.

4.2 Key material

Each user holds two independent keypairs, deliberately separated by function:

- An ordinary **EVM wallet keypair**, used to sign transactions and to authenticate to the backend via `personal_sign`. This key is public-facing and is assumed to be linkable to the user.
- A **Hushh encryption keypair** (`sk_h`, `pk_h`), used only to seal and open note payloads. `pk_h` is published in the username registry; `sk_h` never leaves the client. It is deterministically derived from a signature over a fixed domain-separation string so that it is recoverable from the wallet alone, with no separate backup:

```
# deterministic derivation of the Hushh encryption key
sig    = wallet.personal_sign("hushh-protocol/v1/encryption-key")
seed   = SHA-256(sig)
sk_h   = clamp_x25519(seed)
pk_h   = X25519_base_mul(sk_h)
```

CONSEQUENCE OF THIS DERIVATION

Losing the wallet loses the ability to open *future* notes but not the funds in already-decrypted notes, provided the note secrets were retained. Conversely, a wallet compromise exposes correspondence, not the pool. The separation is intentional: the key that can spend and the key that can read are different keys with different exposure profiles.

4.3 The note

A note is the unit of value in the pool. It exists in three representations, and the distinction between them is where most confusion about shielded pools originates.

REPRESENTATION	CONTENTS	WHERE IT LIVES
Plaintext note	(secret, recipientDigest, amount, token)	Sender's browser at creation; recipient's browser after decryption. Never anywhere else.
Commitment	$C = H_4(\text{secret}, \text{recipientDigest}, \text{amount}, \text{tokenHash})$	On-chain, as an event field and a tree leaf. Reveals nothing about its preimage.
Sealed payload	$\text{Enc}(\text{pk}_h, \text{plaintext note})$	Backend inbox, as opaque ciphertext.

The `recipientDigest` is a field element that binds the note to its intended payout. In v1 it is derived from the recipient's wallet address; the P1 upgrade replaces this with a per-payment stealth digest derived via ECDH, so that repeat payments to the same person no longer share an on-chain tag. The circuit is unchanged by that upgrade because it treats the digest as an opaque public input.

4.4 Algorithms

The protocol is specified as four algorithms. `Setup` runs once per deployment; the rest run once per payment.

```

Setup( $1^\lambda$ )  $\rightarrow$  (pk, vk, addr)
1. (pk, vk)  $\leftarrow$  Groth16.KeyGen( $R_{\text{withdraw}}$ )           # §5.1
2. deploy WithdrawVerifier(vk)
3. deploy MerkleRootManager(window = 64)
4. deploy BulletPool(verifier, rootManager)
5. tree  $T \leftarrow$  empty Poseidon tree, depth 20
6. rootManager.postRoot(root(T))                     # empty-tree root

Deposit(pk $_h^{\text{recipient}}$ , amount, token)  $\rightarrow$  (C, note)
1. secret  $\leftarrow$  $ F                                # 254-bit, client-side CSPRNG
2. recipientDigest  $\leftarrow$  Digest(recipient)
3. tokenHash  $\leftarrow$  uint256(uint160(token))
4.  $C \leftarrow H_4(\text{secret}, \text{recipientDigest}, \text{amount}, \text{tokenHash})$ 
5. assert token  $\in$  BulletPool.registry
6. assert amount  $\in$  DENOMINATIONS
7. ERC20(token).approve(BulletPool, amount)
8. BulletPool.deposit(token, amount, C)               # emits Deposit(C, leafIndex, ...)
9. ct  $\leftarrow$  Enc(pk $_h^{\text{recipient}}$ , (secret, recipientDigest, amount, token))
10. POST /notes { commitment: C, ciphertext: ct }
    # the plaintext note is discarded by the sender after this point

Index()                                              # continuous, off-chain
on Deposit(C, leafIndex) at block b:
1. wait until confirmations(b)  $\geq$  K
2. assert leafIndex = |T|                           # strict ordering; else resync
3. T.insert(C)
4. rootManager.postRoot(root(T))                     # relayer key

Withdraw(note, recipientAddr)  $\rightarrow$  tx
1. ct  $\leftarrow$  GET /notes ; note  $\leftarrow$  Dec(sk $_h$ , ct)   # in browser
2. (path, indices, root)  $\leftarrow$  GET /witness/:C
3. assert rootManager.isKnownRoot(root)
4. nullifier  $\leftarrow$   $H_1(\text{note.secret})$ 
5.  $\pi \leftarrow$  Groth16.Prove(pk, x, w) where
   x = [root, nullifier, recipientDigest, amount, tokenHash]
   w = [secret, path[20], indices[20]]
6. BulletPool.withdraw( $\pi$ , root, nullifier,
   recipientDigest, recipientAddr, token, amount)
7. PATCH /notes/:id/claim                          # UI bookkeeping only

```


THE INVARIANT THAT PRODUCES UNLINKABILITY

Compare the on-chain footprints. `Deposit` publishes $(C, \text{leafIndex}, \text{token}, \text{amount})$ and is signed by the sender. `Withdraw` publishes $(\text{nullifier}, \text{recipient}, \text{token}, \text{amount}, \text{root}, \pi)$ and may be signed by anyone. The intersection of these two sets is $(\text{token}, \text{amount})$ — which is shared with every other note of that denomination — and `root`, which is shared with every note in the tree. *C never appears in a withdraw. The nullifier never appears in a deposit.* There is no field to join on.

4.5 Denominations

Amounts are quantised to a small preset ladder, currently $1 / 10 / 50 / 100$ USDC in base units of 10^6 . Larger payments are composed of several notes, each deposited separately and each withdrawable separately.

The reason is amount-matching resistance. If arbitrary amounts were permitted, a withdrawal of 4,317.62 USDC would identify its deposit uniquely regardless of the cryptography between them; the anonymity set would collapse to one. Quantisation forces every note of a given size into a single indistinguishable class. The cost is a UX quantisation and a larger number of transactions for large payments, and a second-order leak discussed in §9.2: composing an unusual total from an unusual multiset of denominations is itself a fingerprint.

SECTION 05

The withdraw circuit

Only the withdraw path requires a circuit. A deposit is a commitment, an ERC-20 transfer, and an event — all publicly verifiable without zero knowledge. Halving the circuit surface this way is the single largest reduction in audit burden available to a note-pool design.

5.1 The relation

Let R_{withdraw} be the relation containing pairs (x, w) where the public statement x and private witness w are

$$\begin{aligned} x &= (\text{root}, \text{nullifier}, \text{recipientDigest}, \text{amount}, \text{tokenHash}) \\ w &= (\text{secret}, \text{pathElements}[20], \text{pathIndices}[20]) \end{aligned}$$

and $(x, w) \in R_{\text{withdraw}}$ if and only if all three of the following hold simultaneously:

C1 • Correct derivation

$C = H_4(\text{secret}, \text{recipientDigest}, \text{amount}, \text{tokenHash})$. The commitment is the Poseidon hash of the private secret together with the three public note attributes.

C2 • Membership

Hashing C upward along `pathElements` with ordering given by `pathIndices` reproduces `root`. Formally, with $h_0 = C$ and for $i \in [0, 20)$: $h_{i+1} = H_2(\text{indices}[i] ? (\text{pathElements}[i], h_i) : (h_i, \text{pathElements}[i]))$, and $h_{20} = \text{root}$.

C3 • Nullifier binding

$\text{nullifier} = H_1(\text{secret})$, using the *same* secret witnessed in C1. This is what makes double-spending impossible: a note has exactly one nullifier, deterministically, and it cannot be spent under a different one.

The withdraw is accepted on-chain iff a Groth16 proof for this relation verifies against the locked public inputs, the root is inside the acceptance window, and the nullifier is unspent (§6.4).

PRIVATE WITNESS · NEVER REVEALED

secret
pathElements[20]
pathIndices[20]

These values exist only in the recipient's browser memory during proof generation. They are never transmitted, never logged, and not recoverable from the proof.

Groth16

PUBLIC INPUTS · VERIFIED ON-CHAIN · ORDER LOCKED

root must be in the 64-root window
nullifier must be unspent
recipientDigest binds the payout address
amount must match a denomination
tokenHash must be a registered token

The verifier learns that a valid note exists. Not *which* one.

The commitment C is a private intermediate. It is computed inside the circuit and never appears in the withdraw transaction.

Figure 2. Circuit interface. The proof reveals membership without revealing the leaf. Note that C is deliberately absent from the public inputs — publishing it would reconstruct the deposit-to-withdraw link and defeat the entire construction.

5.2 Constraint budget

Constraint count determines proving time, which determines whether client-side proving is viable, which determines whether the "no secret leaves the browser" invariant survives contact with product requirements. The dominant term is the Merkle path.

COMPONENT	COUNT	NOTES
Merkle path, $20 \times \text{Poseidon}_2$	dominant	Roughly 240 constraints per Poseidon ₂ evaluation in the circomlib instantiation, so on the order of 5k constraints total. This is the term to optimise if depth changes.
Path index selectors, $20 \times \text{mux}$	small	Two constraints each for the conditional swap, plus one booleanity constraint per bit.
Commitment, $1 \times \text{Poseidon}_4$	small	Single evaluation.
Nullifier, $1 \times \text{Poseidon}_1$	small	Single evaluation.
Public-input plumbing	negligible	Equality constraints binding computed values to declared signals.

ORDER-OF-MAGNITUDE FIGURES. REPLACE WITH MEASURED VALUES FROM SNARKJS R1CS INFO AFTER ZK:BUILD.

Practically: a circuit in the low thousands of constraints proves in the low seconds in a desktop browser via snarkjs and WebAssembly, and in a small number of seconds on recent mobile hardware. This is the regime the design targets. A Keccak-based Merkle path, at roughly 150k constraints for the same depth, would push proving into minutes and force the proving step onto a server, which would reintroduce the secret-handling trust that the architecture exists to eliminate.

5.3 Public input encoding

All five public inputs are field elements. Two require explicit encoding discipline:

```
tokenHash      = uint256(uint160(tokenAddress))  # 160 bits, always < r
recipientDigest = uint256(uint160(recipientAddr)) # v1; 160 bits, always < r
amount         = raw ERC-20 base units          # e.g. 10_000_000 for 10 USDC
nullifier      = H1(secret)                     # native field element
root           = Poseidon tree root              # native field element
```

Both address-derived values fit in 160 bits and therefore reduce trivially. The contract derives tokenHash identically in BulletPool.tokenHashOf, so the on-chain and in-circuit encodings cannot drift. When the P1 stealth digest replaces the v1 address digest, recipientDigest becomes a full field element derived from an ECDH shared secret, and the reduction must then be performed explicitly and identically on both sides. That change is flagged here because a silent mismatch between an in-circuit reduction and an on-chain one produces proofs that verify against a statement other than the one the contract believes it is checking.

5.4 Soundness-critical checks

Four checks carry the security of the circuit. Each is called out for adversarial testing rather than assumed correct.

CHECK	FAILURE CONSEQUENCE
Index bit booleanity	An unconstrained <code>pathIndices[i]</code> lets a prover interpolate between the two hash orderings at level <code>i</code> and forge membership for a leaf that is not in the tree. This is the most frequently reported bug class in production Merkle circuits and must be tested with a witness containing a non-boolean index.
Nullifier derivation	If the nullifier is not constrained to derive from the <i>same</i> secret as the commitment, a note can be spent repeatedly under fresh nullifiers. This is a direct drain of the pool and is the highest-severity failure in the system.
Public input ordering	The order <code>[root, nullifier, recipientDigest, amount, tokenHash]</code> is locked. A mismatch between the circuit's declared order and the verifier contract's argument order silently changes the statement being proven.
Verifier replacement	The mock verifier used before <code>zk:build</code> accepts everything. Deployment scripts must assert that the live verifier is the generated <code>WithdrawVerifier</code> and not the mock. A status check for this is part of <code>pnpm status</code> .

5.5 Non-malleability and front-running

A Groth16 proof sitting in the mempool is public data. Without countermeasures, an observer could lift it, resubmit it with their own address as the payout target, and steal the note by winning the ordering race. This class of attack has drained real deployments.

Hushh forecloses it structurally rather than procedurally. `recipientDigest`, `amount`, and `tokenHash` are all inside the commitment (C1) and all public inputs of the proof. The contract checks that the `recipient` argument of `withdraw` matches the `recipientDigest` that the proof commits to. Changing the payout address changes a public input, which changes the statement, which invalidates the pairing check. There is nothing to steal: *a proof pays only the address it was generated for*.

The same argument covers value: an observer cannot up-value a withdrawal, because `amount` is likewise bound. And it covers token substitution, because `tokenHash` is bound. The proof is a bearer instrument only in the sense that anyone may *submit* it — which is a feature, since it permits a third-party relay to pay gas on the recipient's behalf without being trusted with anything (§12).

SECTION 06

On-chain design

6.1 Contracts

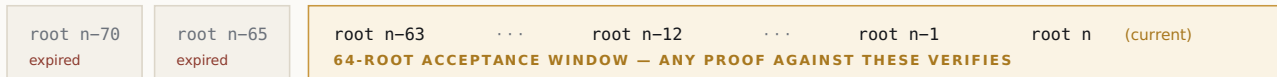
CONTRACT	ROLE	CRITICALITY
<code>BulletPool.sol</code>	Custody of deposited notes; <code>deposit</code> and <code>withdraw</code> entrypoints; the nullifier set; an embedded registry of accepted tokens; pause; <code>Ownable2Step</code> ; reentrancy guard.	Safety-critical. All funds sit here.
<code>MerkleRootManager.sol</code>	Rolling window of the 64 most recent valid roots, with <code>isKnownRoot()</code> consulted at withdraw time.	Safety-critical for soundness of membership.
<code>WithdrawVerifier.sol</code>	Groth16 verifier generated by snarkjs at circuit build; replaces the mock after <code>zk:build</code> .	Safety-critical. Generated, not hand-written.
<code>MockUSDC.sol</code>	Test ERC-20 standing in for USDC during testnet operation.	Testnet only.

Verifier and root manager are swappable by the owner. This is an availability affordance — a circuit rebuild or a root-manager fix should not require migrating custody — and it is also the largest owner-held power in the system. It is recorded honestly in the trust matrix (§8.5): an owner who installs a malicious verifier can drain the pool. Timelocking this replacement is a mainnet prerequisite.

6.2 Why a 64-root window

A naive design accepts only the single current root. That design is broken in practice. Between the moment a recipient fetches their witness and the moment their transaction is mined, any *other* user's deposit advances the root and invalidates their proof. The recipient must re-fetch, re-prove, and resubmit — and loses the race again if the pool is still busy. The failure rate scales with pool activity, meaning the system fails hardest exactly when the anonymity set is strongest and users most want to transact.

ROOT HISTORY — OLDEST TO NEWEST



A recipient who fetched a witness at root `n-12` can still withdraw after twelve further deposits have landed. Without the window, every concurrent deposit would invalidate every in-flight proof.

The window is bounded rather than unbounded so that a root cannot be replayed indefinitely after the indexer set has changed.

Figure 3. The rolling root window decouples proving from block timing.

Window sizing is a straightforward liveness calculation. Let λ_d be the deposit arrival rate and T_p the time from witness fetch to inclusion (proving plus signing plus block time). A proof survives if fewer than W deposits land during T_p . Under a Poisson arrival model the failure probability is

$$P[\text{fail}] = P[N(T_p) \geq W] = 1 - \sum_{k < W} e^{-\lambda_d T_p} (\lambda_d T_p)^k / k!$$

With $W = 64$, failure requires 64 deposits inside a single proving window. At demo volumes this is unreachable; at a sustained one deposit per second and a 20-second proving window, the expected count is 20 and failure remains rare. The parameter should be revisited if sustained deposit rate ever approaches W / T_p , and the cost of increasing it is bounded storage plus a slightly longer replay horizon.

6.3 Withdraw validation order

The pool rejects the transaction unless every check passes, in this order. The ordering is not cosmetic.

```
1. isKnownRoot(root)           # root is inside the 64-entry window
2. !nullifiers[nullifier]      # the note has not already been spent
3. verifyProof(π, publicInputs) # Groth16 pairing check, locked input order
4. registry[token] && amount ∈ DENOMINATIONS
5. recipientDigest binds recipient
   --- effects ---
6. nullifiers[nullifier] = true # mark spent BEFORE the transfer
7. ERC20(token).transfer(recipient, amount)
8. emit Withdrawal(nullifier, recipient)
```

Cheap checks precede the expensive pairing check so that malformed transactions fail early and cheaply. More importantly, step 6 precedes step 7: the nullifier is written before value moves, so a reentrant callback from a hostile token encounters a note that is already spent. This is checks-effects-interactions applied to the one state transition that matters, and it is enforced by a reentrancy guard as well, on the principle that the ordering argument should not be the only thing standing between the pool and a drain.

6.4 Gas profile

OPERATION	DOMINANT COST	INDICATIVE
deposit	ERC-20 <code>transferFrom</code> , one storage write for the leaf counter, one event.	~60–90k gas
withdraw	Groth16 pairing check via BN254 precompiles, one storage write for the nullifier, ERC-20 <code>transfer</code> , one event.	~280–350k gas
postRoot	One storage write into the ring buffer. Paid by the relayer, not the user.	~30–45k gas

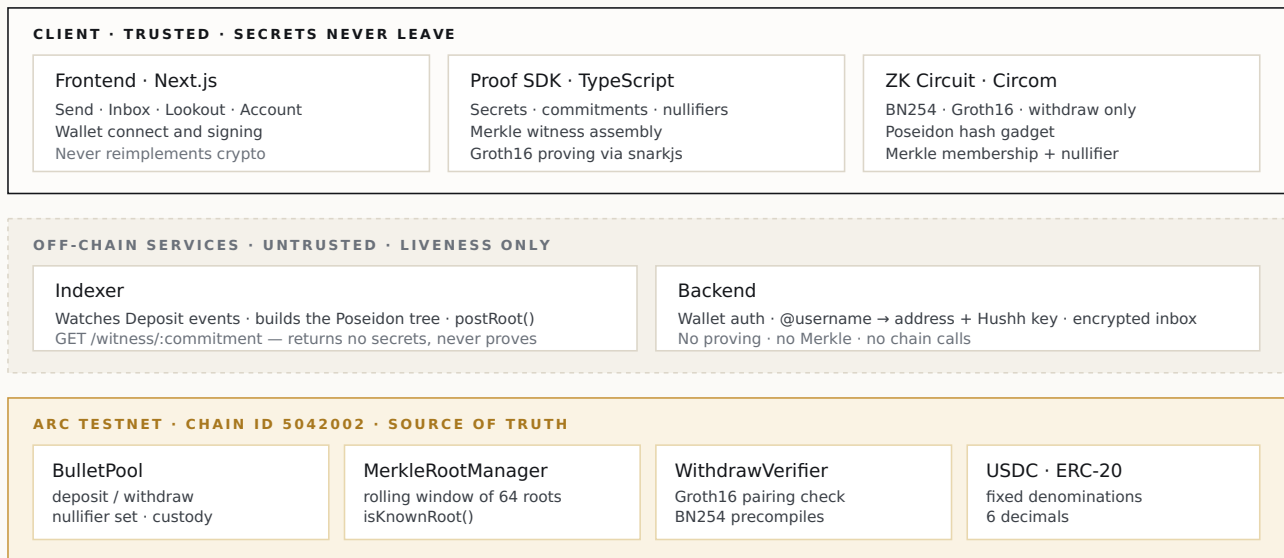
INDICATIVE RANGES BASED ON TYPICAL BN254 GROTH16 VERIFICATION COSTS. REPLACE WITH MEASURED VALUES FROM A HARDHAT GAS REPORT AGAINST THE DEPLOYED BUILD.

The economically relevant observation is that `withdraw` is a *constant* cost, independent of pool size. A pool with a million notes verifies as cheaply as one with ten. Privacy therefore does not become more expensive as it becomes stronger — an unusual and favourable property, and the reason Groth16 was chosen over proof systems with better setup characteristics but logarithmic or linear verification.

SECTION 07

Off-chain infrastructure

Six layers, with a hard boundary between what runs on the client and what runs on a server. Every secret lives above the boundary. Every server below it is a liveness dependency and never a safety one.



Downward arrows carry proofs and public data only. No arrow in this diagram carries a secret.

Figure 4. Layered architecture and trust zones. The boundary between the solid box and the dashed box is the security boundary of the entire system.

7.1 Indexer

The indexer watches `Deposit` events, inserts leaves in strict `leafIndex` order, and posts the resulting root using a dedicated relayer key. It exposes `/health`, `/root`, `/witness/:commitment`, `/deposit/:commitment`, and `/stats`. It never returns a secret and never generates a proof.

Three operational properties matter more than they appear to:

- **Strict ordering.** The indexer asserts `leafIndex = |T|` before every insert. A gap means events were missed, and the correct response is to resync from the chain rather than to insert out of order, which would produce a root no honest party can reproduce.
- **Confirmation depth.** Leaves are finalised only after `K` confirmations, so a reorg cannot bake a reverted deposit into the tree. Arc's fast deterministic finality keeps `K` small and therefore keeps deposit-to-withdrawable latency short.
- **Crash recovery.** Tree state is persisted, and on restart the indexer replays from the last finalised leaf. The tree is fully reconstructible from chain events alone, which is what makes the next property possible.

WHY THE INDEXER CANNOT FORGE A LEAF

Every leaf is derived from a public `Deposit` event. Any party — a user, a watchdog, a competitor — can independently rebuild the tree from the event log and check that a posted root matches. A malicious indexer that inserts a fabricated leaf publishes a root inconsistent with the chain's own history, and that inconsistency is detectable by anyone who looks. The indexer's real power is *withholding*: it can decline to post roots or to serve witnesses, stalling withdrawals. That is a liveness attack, and §12 addresses it by making root posting permissionless.

7.2 Backend and the usability layer

Steps 1–4 of §4 are sufficient for privacy and insufficient for adoption. Three components carry the user experience, and the design constraint on all three is that none may be trusted with funds or secrets.

LAYER	FUNCTION	WHAT IT CAN SEE
Username registry	Resolves <code>@handle</code> to a wallet address and a Hushh encryption public key, so senders address humans rather than hex.	Public directory data only.
Encrypted inbox	Note payloads sealed to the recipient's Hushh public key, stored server-side as opaque ciphertext and delivered on login.	Ciphertext and routing metadata: who messaged whom, and when.
Proof SDK	Generates secrets, commitments and nullifiers; assembles Merkle witnesses; produces Groth16 proofs entirely in the browser.	Runs client-side only. Sees everything, transmits nothing.

BACKEND INVARIANT

The backend stores ciphertext and routing metadata. It never holds a secret, never generates a proof, and cannot spend a note. **A total compromise of the Hushh backend leaks correspondence metadata. It does not leak funds and does not break on-chain unlinkability.** This sentence is the design constraint every backend feature is checked against; a feature that would violate it is not built.

7.3 Note encryption

The sealed payload uses ephemeral-static X25519 key agreement with an AEAD, which gives sender anonymity within the ciphertext itself and forward secrecy against future compromise of the ephemeral key:

```
(esk, epk) ←$ X25519 keypair           # fresh per note
shared    = X25519(esk, pk_h^recipient)
key       = HKDF-SHA256(shared, info="hushh/v1/note")
ct        = XChaCha20-Poly1305.Seal(key, nonce, note)
payload   = epk || nonce || ct
```

The recipient scans their inbox, attempts decryption with `sk_h`, and keeps what opens. Note that the payload carries no sender identifier: the backend learns who *posted* a ciphertext from the authenticated session, but the ciphertext itself does not name a sender. A user who prefers not to leak that routing metadata can bypass the inbox entirely and deliver the payload out of band by copy-paste, which the SDK supports and which costs nothing in on-chain privacy.

KEY SUBSTITUTION, STATED PLAINLY

A malicious registry could serve its own `pk_h` in place of the intended recipient's, and thereby read note payloads. It could not spend those notes, because spending is gated by the on-chain proof against `recipientDigest`. For high-value payments, verify the counterparty's key out of band. The v2 on-chain identity registry (§12) removes this seam by moving handle-to-key binding on-chain, where substitution is public and detectable.

7.4 SDK discipline

One rule, enforced without exception: **the frontend calls the SDK and never reimplements cryptography**. Poseidon, commitment derivation, nullifier derivation, witness assembly, and proving exist in exactly one place. A fix lands once, an audit has one target, and no UI change can silently alter a security-relevant computation. Convenience reimplementations in application code are the mechanism by which correct protocols become incorrect products.

SECTION 08

Security analysis

The question worth asking of any privacy system is not "is it encrypted" but "what does each party gain by defecting". This section defines the properties formally, sketches why they hold, and then answers that question party by party.

8.1 Security properties

P1 • Completeness

For every honestly generated note whose commitment has been inserted under a root still inside the acceptance window, the recipient can produce a proof that `withdraw` accepts. Follows from Groth16 completeness and the window-sizing argument of §6.2.

P2 • Balance (no theft, no inflation)

No probabilistic polynomial-time adversary can cause the pool to pay out more than the total deposited, except with probability $\text{negl}(\lambda)$. Equivalently: each accepted `withdraw` can be mapped injectively to a distinct prior `deposit` of the same token and amount.

Sketch. An accepting proof yields, by knowledge soundness (A1), an extractable witness `(secret, path, indices)` satisfying C1–C3. C2 places `C = H4(secret, ·)` under a root in the window. Roots are posted only by insertion of observed `Deposit` commitments, so `C` corresponds to a real deposit unless the adversary found a Poseidon collision (A2) or the setup was compromised (A3). Injectivity follows from C3 plus the nullifier set: two accepted withdrawals mapping to the same deposit would require the same `secret`, hence the same `H1(secret)`, hence a repeat nullifier, which step 2 of §6.3 rejects.

P3 • Double-spend resistance

A note can be withdrawn at most once. Immediate from P2's injectivity argument: the nullifier is a deterministic function of the secret, so a second spend necessarily collides with the first in the spent set.

P4 • Deposit-withdraw unlinkability

Consider an adversary who observes the full chain, chooses two honest deposits D_0 and D_1 of the same token and denomination, and is then shown a withdrawal spending D_b for uniform b . The adversary's advantage in guessing b is negligible with respect to on-chain data alone.

Sketch. The withdraw transaction publishes $(\text{root}, \text{nullifier}, \text{recipientDigest}, \text{amount}, \text{tokenHash}, \pi)$. Of these, amount and tokenHash are equal for D_0 and D_1 by construction, and root contains both. $\text{nullifier} = H_1(\text{secret})$ is independent of the commitment's other inputs and, under A2, is unlinkable to C without secret . π is zero-knowledge, so it is simulatable from x alone. Hence the adversary's view is identically distributed for $b = 0$ and $b = 1$.

The qualifier "on-chain data alone" is doing real work. P4 says nothing about timing correlation, network-level observation, off-chain metadata, or the size of the candidate set. Those are treated quantitatively in §9 and honestly in §10.

P5 • Payout binding (non-malleability)

A proof observed before inclusion cannot be modified to pay a different address, a different amount, or a different token. Immediate from §5.5: all three are public inputs bound inside the commitment, so altering any of them invalidates the pairing check.

8.2 What each party can do

PARTY	STEAL	CENSOR	DEANONYMISE	NOTES
Indexer / relay	No	Yes	No	Can withhold roots or witnesses, stalling withdrawals indefinitely. Cannot forge a leaf: the pool's own <code>Deposit</code> log constrains what any honest reconstruction of the tree can contain, and inconsistency is publicly detectable.
Backend	No	Yes	Partial	Holds ciphertext and routing metadata. Learns who messages whom and when. Does not learn amounts, secrets, or the on-chain link. Can refuse to serve the inbox, which is why local note export exists.
Contract owner	No*	Registry	No	Can add or remove accepted tokens and pause. *Can replace the verifier, and a malicious verifier drains the pool. This is the largest owner power in the system and is the reason a timelock is a mainnet prerequisite.
Trusted setup	If retained	No	No	Toxic-waste retention permits forged proofs, which is theft from the pool. It does <i>not</i> permit deanonymising users — a critical asymmetry, since it means a bad ceremony risks funds but not the safety of people who used the system for privacy.
Chain validators	No	Yes	No	Standard L1 censorship assumptions. Proof contents remain opaque to block producers.
Network observer	No	No	Timing	Can correlate deposit and withdrawal timing in a thin pool, and can correlate IP addresses across deposit and withdraw if the user does not change network. Mitigated by pool depth, by delay, and by transport-level precautions.

THE INVARIANT THAT MATTERS

Custody and unlinkability rest exclusively on the contracts and the circuit. Every off-chain component is a liveness dependency. A malicious indexer and a malicious backend, *colluding*, can stop Hushh from working. They cannot take a user's funds and cannot reconstruct the deposit-to-withdraw link.

8.3 Attack catalogue

ATTACK	MITIGATION
Double withdrawal	The nullifier is deterministically derived from the secret inside the circuit. Two withdrawals of one note necessarily produce the same nullifier, and the second is rejected by the spent-set check before the pairing check even runs.
Proof front-running	<code>recipientDigest</code> is a locked public input bound inside the commitment. A proof lifted from the mempool pays only the address it was generated for; redirecting it invalidates the pairing check (§5.5).
Forged Merkle root	Roots are constrained by observable on-chain <code>Deposit</code> events. Any party can independently rebuild the tree and detect a root inconsistent with the event log. Detection is public; v2 makes posting permissionless so detection also has a remedy.
Reentrancy on withdraw	The nullifier is written before the ERC-20 transfer, so a reentrant callback from a hostile token encounters an already-spent note. A reentrancy guard is applied in addition, not instead.
Amount fingerprinting	Fixed denominations only. A withdrawal of a non-standard amount — the classic correlation vector — is not expressible in the system.
Backend key substitution	A malicious registry could serve its own encryption key and intercept ciphertext, though not funds. Verify keys out of band for high-value payments; the v2 on-chain registry closes the seam (§7.3).
Malicious token in registry	Only owner-registered tokens are accepted. A fee-on-transfer or rebasing token would break the fixed-denomination invariant, so the registry admits only standard-behaviour ERC-20s. This is an owner responsibility and is listed as such.
Weak client randomness	The note secret is sampled from the browser CSPRNG. A predictable secret is a stealable note, and this is entirely a client-side property — no amount of on-chain design compensates for it. The SDK is the only place secrets are generated, which is why §7.4 forbids reimplementing.
Denial by root withholding	Currently unmitigated: a stalled relayer blocks all withdrawals. Funds are not at risk and the stall ends when posting resumes. Permissionless posting (v2) is the structural fix.
Dust and probe deposits	An adversary who deposits many notes to inflate their share of the anonymity set reduces the effective set for everyone else (§9.1). This is inherent to pooled anonymity and is mitigated only by genuine volume.

SECTION 09

Anonymity as a quantity

P4 is a statement about cryptography. What a user actually experiences is a number: how many other notes could plausibly have been the source of their withdrawal. That number is smaller than the pool size, and it decays with time. Users deserve the formula, not a reassurance.

9.1 The effective anonymity set

Naively, the anonymity set is every unspent note in the pool. Three effects shrink it:

Denomination partitioning. A 50 USDC withdrawal can only have come from a 50 USDC deposit. The pool is partitioned by `(token, denomination)` and a user's set is their partition, not the whole tree. A ladder of four denominations across two tokens partitions the pool eight ways.

Ordering. Only notes deposited *before* the withdrawal are candidates.

Adversary-owned notes. An observer who deposited some of the notes in the partition knows those are not the source. If an adversary controls a fraction `f` of the partition, the honest set shrinks by that fraction.

Writing `N_d(t)` for the number of unspent notes of denomination `d` at time `t`, the effective set is

$$A_{\text{eff}} = (1 - f) \cdot N_d(t)$$

and the observer's uncertainty is properly measured not by the count but by the Shannon entropy of their posterior over candidates:

$$H = - \sum_i p_i \log_2 p_i \quad \text{bits, with } H = \log_2 A_{\text{eff}} \text{ only when the posterior is uniform}$$

The posterior is uniform only if the observer has no timing information. They almost always have timing information.

9.2 Timing decay

Suppose an observer models the delay between deposit and withdrawal as a distribution $g(\Delta)$. Seeing a withdrawal at time t , they assign each candidate deposit at time t_i the weight $p_i \propto g(t - t_i)$. If users withdraw promptly, g is sharply peaked and the posterior concentrates on a handful of recent deposits — the entropy collapses even though the raw count is large.

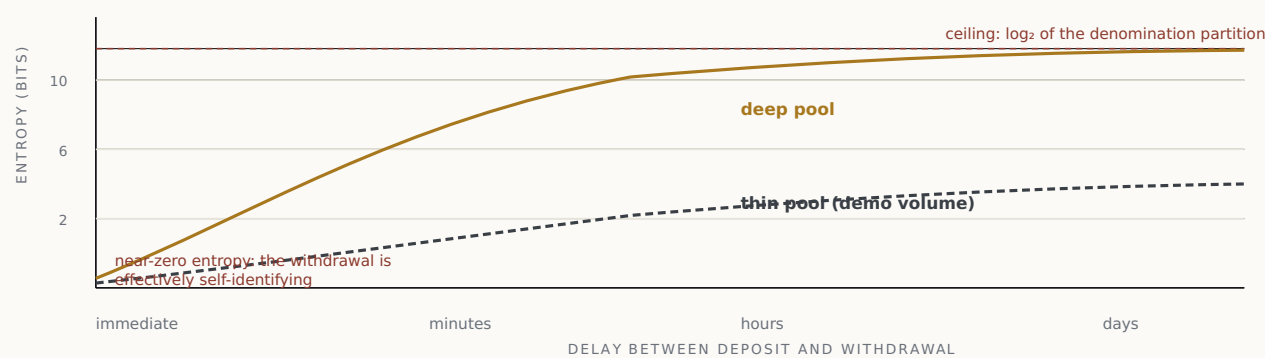


Figure 5. Qualitative shape of observer uncertainty against withdrawal delay. Waiting is not a decoration; it is the single most effective control a user has, and it is free. In a thin pool no amount of waiting reaches a strong guarantee — the curve is bounded above by the size of the denomination partition.

9.3 What this means for a user

CONTROL	EFFECT
Wait before withdrawing	Flattens the observer's timing posterior. The strongest free control available. Withdrawing seconds after a deposit lands is close to publishing the link.
Use common denominations	Keeps you in the largest partition. An unusual denomination is a small partition and therefore a small set, regardless of the tree's total size.
Vary the composition of large payments	Paying 230 USDC as $2 \times 100 + 3 \times 10$ at one instant is a recognisable multiset. Spreading composition over time and mixing sizes blunts it.
Withdraw to a fresh address	Prevents the payout address itself from re-linking the payment to an identity that the pool just spent effort concealing.
Separate network context	The protocol conceals the on-chain link, not the IP address. Depositing and withdrawing from the same network fingerprint gives an observer a correlation the chain does not.
Deliver out of band	Skipping the inbox for copy-paste delivery removes the backend's view of correspondence metadata at the cost of manual handling.

THE UNCOMFORTABLE SUMMARY

At demo-stage volume the anonymity set is thin and a determined observer with timing data can narrow it substantially. The cryptography is not the limiting factor and improving it would not help. Volume is the limiting factor. This is a property of every pooled-anonymity system, and any project claiming otherwise at launch volume is misrepresenting the mathematics.

SECTION 10

Honest limits

Stated without hedging, because users calibrate real-world risk against these sentences. Each item is a property Hushh v1 does not have.

— **Unlinkability, not confidentiality.** Deposit and withdraw are unlinkable. Amounts are standardised into fixed denominations, not encrypted. A 50 USDC note is a visible 50 USDC ERC-20 transfer. This is not a full shielded pool with hidden balances.

- **Anonymity scales with pool size.** A thin pool is a weak anonymity set. At demo-stage volume the set is small, and a determined observer with timing data may narrow it substantially (§9).
- **Root posting is permissioned.** A single relayer posts roots today. It cannot steal and cannot deanonymise, but it can stall withdrawals indefinitely.
- **The trusted setup is single-contributor.** A retained toxic-waste parameter would permit forged proofs — theft from the pool, not deanonymisation of users. An MPC ceremony is a prerequisite for mainnet.
- **The owner can replace the verifier.** Without a timelock, this is an unmitigated path to draining the pool by an owner or by whoever compromises the owner key. It is listed here rather than buried in §8.2 because it is the sharpest single-party risk in the current deployment.
- **Off-chain metadata leaks.** Claim links, email delivery, and server logs can leak timing and correspondence patterns even while on-chain unlinkability holds fully. A subpoena to the backend operator returns a correspondence graph.
- **Network-level correlation is unaddressed.** The protocol conceals the on-chain link. It does nothing about IP addresses, RPC provider logs, or browser fingerprints, and an observer at that layer may reconstruct what the chain does not reveal.
- **Timing correlation is real.** A deposit followed minutes later by a same-denomination withdrawal in an otherwise quiet pool is a weak but genuine signal. Waiting is a meaningful defence and is the user's most effective control.
- **Notes are bearer instruments.** A lost secret is a lost note, permanently, with no recovery path and nobody to appeal to. The encrypted inbox mitigates loss for registered users; it does not eliminate it.
- **The circuit is unaudited.** No third-party review has been completed. Treat all deployed code as experimental.

PLAINLY

Hushh is testnet software with an unaudited circuit and a single-contributor trusted setup. Do not use it to protect information whose disclosure you cannot afford.

SECTION 11

Deployment

NETWORK PARAMETER	VALUE
Network	Arc Testnet
Chain ID	5042002
Currency	USDC
RPC	https://rpc.testnet.arc.network
Explorer	https://testnet.arcsan.app
CONTRACT	ADDRESS
BulletPool	0x9E65584C6ACE76cEFde4FD5f50E6bB9763999dCe
MerkleRootManager	0x6D6530A1fC908792DA2aa533a5F299283E6F062a
BulletVerifier	0x0A95c430Bf6AA3140A4Bf04C8D5982472331008d
MockUSDC	0x7d7af5715e5671e0E3126b2428Dc2629bd9061e3
SERVICE	ENDPOINT
App	hush-protocol-zeta.vercel.app
Backend	hush-protocol-backend.onrender.com
Indexer	hush-protocol-indexer.onrender.com

Build requirements: Node 22+, pnpm 9+, Circom for circuit builds. The verifying key `withdraw_vk.json` is published so that any party can independently verify a proof off-chain and confirm that the deployed verifier corresponds to the published circuit. Intermediate powers-of-tau and

zkey material is never committed.

SECTION 12

Roadmap

The sequence is deliberate: decentralise the components that can stall the system before adding features that widen its surface area. A privacy protocol that grows features faster than it sheds trusted parties ends up with a large attack surface defended by a small number of promises.

RELEASE	THEME	CONTENTS
v1 SHIPPED	Core rail	Fixed-denomination USDC notes · Groth16 withdraw circuit · 64-root acceptance window · username resolution · encrypted inbox · browser-native proof SDK.
v2 NEXT	Decentralise the edges	Permissionless root posting with multiple independent watchers · MPC trusted setup ceremony · a relayer network enabling gasless withdrawal for recipients holding no gas · client-side witness reconstruction so a user can withdraw with no indexer at all · on-chain identity registry, closing the key-substitution seam (§7.3) · timelocked verifier replacement.
v3	Richer privacy	Variable amounts via encrypted balances (Pedersen commitments plus range proofs) · note splitting and merging · multi-token shared anonymity sets · stealth digests as the default for every send, giving repeat-payment unlinkability · recurring shielded payroll.
v4	Compliance optionality	Viewing keys for voluntary disclosure · selective audit trails · proof-of-innocence style attestations. Disclosure controlled by the user, never by the protocol.

Two items deserve emphasis because they change the trust model rather than the feature set. **Client-side witness reconstruction** removes the indexer from the critical path entirely: a user who syncs `Deposit` events themselves needs no witness API, which converts the indexer from a liveness dependency into a convenience. **Permissionless root posting** means a stalled relayer is a nuisance rather than an outage. Together they eliminate the only two ways the current system can be stopped.

SECTION 13

Compliance posture

Financial privacy and financial crime are not the same subject, and conflating them produces both bad policy and bad protocols. Hushh's position is specific:

- **Non-custodial by construction.** No party in the system, including its authors, can move a user's funds. There is no balance to freeze and no account to seize because there is no account.
- **Disclosure is a user capability, not a protocol backdoor.** The v4 viewing key design lets a user prove to a counterparty, an auditor, or a court that a specific withdrawal corresponds to a specific deposit — voluntarily, selectively, and without granting ongoing access. Because the note secret is the root of that proof, only the user can produce it. A protocol-level disclosure mechanism, by contrast, is a key that exists whether or not anyone consents to its use, and is therefore not built.
- **Everything is public except the link.** Deposits, withdrawals, amounts, and total pool value are all on-chain and continuously auditable. What is hidden is specifically the correspondence between two public events.
- **Denomination ceilings and integration-layer controls** are where jurisdiction-specific requirements belong. An operator building a payroll product on Hushh can apply their own controls at the point of onboarding without the protocol encoding any particular jurisdiction's rules into a contract that cannot be changed.

None of this is legal advice, and the regulatory treatment of shielded pools varies substantially by jurisdiction and is unsettled in several major ones. Integrators should take their own advice.

SECTION 14

Design principles and open problems

14.1 Principles

PRINCIPLE	WHY IT IS ENFORCED
Never reimplement crypto in the app	The frontend calls the SDK. Poseidon and proving exist in exactly one place, so a fix lands once and an audit has one target.
The server holds ciphertext, never secrets	A full backend compromise must leak metadata only. Never funds, never identities, never the on-chain link.
Safety on-chain, liveness off-chain	Anything off-chain may stall the system. Nothing off-chain may break it. This is the line every architectural decision is checked against.
State the limits	Users cannot calibrate real-world risk against marketing language. Section 10 exists for that reason and is maintained as a first-class part of this document, not an appendix to it.
Prefer a smaller circuit	Every constraint is code that must be correct and cannot be patched after notes exist under it. Deposit requires no circuit, and that decision halved the audit surface at no cost to the guarantee.

14.2 Open problems

- **Bootstrapping an anonymity set.** Privacy is only valuable at volume and volume only arrives once privacy is valuable. Whether subsidised early deposits, shared multi-token sets, or integration into a high-frequency payment flow is the right answer is unresolved, and it is the hardest problem in the project.
- **Withdrawal timing as a protocol concern.** Users are advised to wait (§9.3), but advice is a weak mechanism. Protocol-enforced randomised delays, batched withdrawal windows, or relayer-side jitter would convert a user discipline into a system property, at some cost to responsiveness.
- **Gas without deanonymisation.** A recipient withdrawing to a fresh address needs gas at that address, and funding it re-links. The v2 relayer network solves this, but a relayer that observes proofs learns timing; relayer selection and payment are themselves a metadata channel.
- **Denomination ladder design.** Fewer sizes means larger partitions and better privacy; more sizes means fewer transactions per payment. The optimal ladder depends on the empirical distribution of payment sizes, which is not yet observable.
- **Recovery without custody.** Social recovery of note secrets without introducing a party who can spend them remains unsolved in a way that ordinary users would accept.

APPENDIX A

Notation

SYMBOL	MEANING
$\mathbb{F}, r$	Scalar field of BN254 and its prime order.
H_1, H_2, H_4	Poseidon hash at arity 1, 2, and 4 over $\mathbb{F}$ (circomlib instantiation).
secret	Uniform field element sampled client-side. Sole spending authority for a note.
C	Note commitment, $H_4(\text{secret}, \text{recipientDigest}, \text{amount}, \text{tokenHash})$. Appears on-chain at deposit and never at withdraw.
nullifier	$H_1(\text{secret})$. Appears on-chain at withdraw and never at deposit.
recipientDigest	Field element binding the note to its payout target. v1: address-derived. P1: stealth-derived per payment.
tokenHash	$\text{uint256}(\text{uint160}(\text{tokenAddress}))$. Derived identically in circuit and contract.
root	Root of the depth-20 Poseidon Merkle tree of commitments.
W	Root acceptance window size. Currently 64.
d	Tree depth. Currently 20, giving capacity $2^{20} = 1,048,576$ notes.
(sk_h, pk_h)	Hushh encryption keypair, X25519, derived deterministically from a wallet signature.
A_eff	Effective anonymity set: unspent notes of the same denomination, excluding adversary-owned notes.

APPENDIX B

Contract surface

```

BulletPool
deposit(token, amount, commitment) → leafIndex
  emits Deposit(commitment, leafIndex, token, amount)

withdraw(proof, root, nullifier, recipientDigest,
  recipient, token, amount)
  emits Withdrawal(nullifier, recipient)

addToken(token) / removeToken(token) / isSupported(token)
tokenHashOf(token) → uint256
pause() / unpause() # Ownable2Step
setVerifier(addr) / setRootManager(addr) # timelock: v2

MerkleRootManager
postRoot(root) # relayer only in v1
isKnownRoot(root) → bool # rolling window of 64

WithdrawVerifier
verifyProof(a, b, c, publicInputs[5]) → bool

```

Observe what is absent from the surface: there is no administrative withdrawal, no balance mapping keyed by user, and no function that accepts a commitment at withdraw time. The absence of the third is what makes the construction work; the absence of the first two is what makes it non-custodial.

APPENDIX C

Public input encoding

```
publicInputs[0] = root           # field element, native
publicInputs[1] = nullifier      # field element, native
publicInputs[2] = recipientDigest # uint160 → field, v1
publicInputs[3] = amount         # ERC-20 base units, e.g. 10_000_000
publicInputs[4] = tokenHash      # uint256(uint160(token))
```

```
# ORDER IS LOCKED. The circuit's declared signal order and the
# verifier contract's argument order must match exactly. A silent
# permutation changes the statement being proven without any
# visible failure until it is exploited.
```

APPENDIX D

Parameters

PARAMETER	VALUE	RATIONALE / CHANGE COST
Curve	BN254 (alt_bn128)	Native EVM precompiles. Changing requires a new verifier and new setup.
Proof system	Groth16	Constant proof size and verification cost. Changing requires a full circuit rebuild.
Hash	Poseidon (circomlib)	Arithmetisation-friendly. Changing invalidates every existing commitment.
Tree depth d	20	1,048,576 note capacity. Deliberate over-provision: cheap now, migration-expensive later.
Root window w	64	Liveness parameter (§6.2). Cheap to raise; raising extends the replay horizon.
Denominations	1 / 10 / 50 / 100 USDC	Amount-matching resistance. Adding sizes fragments the anonymity set.
Confirmations K	chain-dependent	Reorg safety before leaf finalisation. Small on Arc.
Note encryption	X25519 + XChaCha20-Poly1305	Ephemeral-static agreement per note. Application layer; changeable without touching the pool.

REFERENCES

Selected references

- Ben-Sasson, Chiesa, Garman, Green, Miers, Tromer, Virza. *Zerocash: Decentralized Anonymous Payments from Bitcoin*. IEEE S&P, 2014.
- Groth. *On the Size of Pairing-Based Non-Interactive Arguments*. EUROCRYPT, 2016.
- Grassi, Khovratovich, Rechberger, Roy, Schofnegger. *Poseidon: A New Hash Function for Zero-Knowledge Proof Systems*. USENIX Security, 2021.
- Pertsev, Semenov, Storm. *Tornado Cash Privacy Solution*. 2019.
- Barbulescu, Duquesne. *Updating Key Size Estimations for Pairings*. Journal of Cryptology, 2019.
- Kim, Barbulescu. *Extended Tower Number Field Sieve*. CRYPTO, 2016.
- Bowe, Gabizon, Miers. *Scalable Multi-party Computation for zk-SNARK Parameters in the Random Beacon Model*. 2017.
- Wahby, Boneh, et al. *ERC-5564: Stealth Addresses*. Ethereum Improvement Proposals.
- Bernstein. *Curve25519: New Diffie-Hellman Speed Records*. PKC, 2006.
- iden3. *circom* and *circomlib*. Reference implementations of the Poseidon and Merkle gadgets used here.

Hushh Protocol · Whitepaper v1.0 · Arc Testnet. This document supersedes the Hushh Litepaper v1.0 and is maintained alongside the implementation; where the two disagree, the implementation and its tests are authoritative and this document is a bug.

Hushh Protocol is testnet software with an unaudited circuit and a single-contributor trusted setup. Do not use it to protect information whose

disclosure you cannot afford. Nothing in this document is financial, legal, or security advice.